21. Operation 8 - Remove by Value

We can also remove a specific value.

numbers.remove(Integer.valueOf(30));

Why do we use:

Integer.valueOf(30)

instead of simply:

numbers.remove(30);

Because with an Integer LinkedList, remove(30) is interpreted as removing the element at **index 30**, not necessarily the value 30.

For beginners, it is clearer to teach:

numbers.remove(Integer.valueOf(30));

when specifically removing an integer value.

Example:

```
import java.util.LinkedList;


public class Main {


  public static void main(String[] args) {


    LinkedList<Integer> numbers = new LinkedList<>();


    numbers.add(10);

    numbers.add(20);

    numbers.add(30);

    numbers.add(40);
```

```
        numbers.remove(Integer.valueOf(30));

        System.out.println(numbers);
    }
}
```

Output:

[10, 20, 40]

22. Easier Example with Strings

With Strings, removing a value is easier to understand.

```
LinkedList<String> names = new LinkedList<>();

names.add("Ana");
names.add("Ben");
names.add("Carlo");

names.remove("Ben");

System.out.println(names);
```

Output:

[Ana, Carlo]

23. Operation 9 - removeFirst()

To remove the first element:

removeFirst()

Example:

```
import java.util.LinkedList;


public class Main {


  public static void main(String[] args) {


    LinkedList<Integer> numbers = new LinkedList<>();


    numbers.add(10);
    numbers.add(20);
    numbers.add(30);


    numbers.removeFirst();


    System.out.println(numbers);
  }
}
```

Output:

[20, 30]

24. Operation 10 - removeLast()

To remove the last element:

removeLast()

Example:

numbers.removeLast();

If we have:

[10, 20, 30]

the result becomes:

[10, 20]

25. Operation 11 - Search Using contains()

To check whether a value exists:

contains()

Example:

```
import java.util.LinkedList;


public class Main {


  public static void main(String[] args) {


    LinkedList<Integer> numbers = new LinkedList<>();


    numbers.add(10);
    numbers.add(20);
    numbers.add(30);
```

```
        if (numbers.contains(20)) {
            System.out.println("20 is found.");
        } else {
            System.out.println("20 is not found.");
        }
    }
}
```

Output:

20 is found.

26. Understanding contains()

This:

numbers.contains(20)

asks:

"Does the Linked List contain 20?"

The answer is either:

true

or:

false

That's why we can use it with:

if

27. Search for a Missing Value

```
if (numbers.contains(50)) {
   System.out.println("50 is found.");
} else {
   System.out.println("50 is not found.");
}
```

If the list is:

[10, 20, 30]

Output:

50 is not found.

28. Operation 12 - Find the Position Using indexOf()

We can find the index of an element using:

indexOf()

Example:

```
int position = numbers.indexOf(20);
```

If:

[10, 20, 30]

then:

indexOf(20)

returns:

1

29. Complete indexOf() Example

```
import java.util.LinkedList;
```

```
public class Main {

  public static void main(String[] args) {

    LinkedList<Integer> numbers = new LinkedList<>();

    numbers.add(10);
    numbers.add(20);
    numbers.add(30);

    int position = numbers.indexOf(20);

    System.out.println("Position: " + position);
  }
}
```

Output:

Position: 1

30. Operation 13 - Get the Size

Use:

size()

Example:

```
import java.util.LinkedList;

public class Main {
```

```
  public static void main(String[] args) {

    LinkedList<Integer> numbers = new LinkedList<>();

    numbers.add(10);
    numbers.add(20);
    numbers.add(30);

    System.out.println("Number of elements: " + numbers.size());
  }
}
```

Output:

Number of elements: 3

31. Understanding size()

If:

[10, 20, 30, 40]

then:

numbers.size()

returns:

4

Remember:

size() tells us how many elements are currently inside the Linked List.

32. Operation 14 - Check if Empty

Use:

isEmpty()

Example:

```
if (numbers.isEmpty()) {
   System.out.println("The list is empty.");
} else {
   System.out.println("The list has elements.");
}
```

If the list is:

[]

output:

The list is empty.

If it contains:

[10, 20]

output:

The list has elements.

33. Operation 15 - Display Using a for Loop

Although Java can simply print:

System.out.println(numbers);

students should also learn how to traverse a Linked List.

We can use a loop.

```
for (int i = 0; i < numbers.size(); i++) {

   System.out.println(numbers.get(i));
```

}

Example:

```
import java.util.LinkedList;

public class Main {

  public static void main(String[] args) {

    LinkedList<Integer> numbers = new LinkedList<>();

    numbers.add(10);
    numbers.add(20);
    numbers.add(30);

    for (int i = 0; i < numbers.size(); i++) {

      System.out.println(numbers.get(i));
    }
  }
}
```

Output:

```
10
20
30
```

34. Understanding the Loop

Suppose:

[10, 20, 30]

First iteration

i = 0

Then:

numbers.get(0)

returns:

10

Second iteration

i = 1

Then:

numbers.get(1)

returns:

20

Third iteration

i = 2

Then:

numbers.get(2)

returns:

30

35. Display Using Enhanced for Loop

For beginners, this is even easier:

for (int number : numbers) {

```
    System.out.println(number);
}
```

Complete example:

```
import java.util.LinkedList;

public class Main {

  public static void main(String[] args) {

    LinkedList<Integer> numbers = new LinkedList<>();

    numbers.add(10);
    numbers.add(20);
    numbers.add(30);

    for (int number : numbers) {

      System.out.println(number);
    }
  }
}
```

Output:

```
10
20
30
```

36. A Complete Beginner Program

Now let's combine the basic operations.

```
import java.util.LinkedList;

public class Main {

  public static void main(String[] args) {

    LinkedList<Integer> numbers = new LinkedList<>();

    // Add elements
    numbers.add(10);
    numbers.add(20);
    numbers.add(30);

    System.out.println("Original List:");
    System.out.println(numbers);

    // Add at beginning
    numbers.addFirst(5);

    System.out.println("After adding 5 at beginning:");
    System.out.println(numbers);

    // Add at end
    numbers.addLast(40);
```

```
        System.out.println("After adding 40 at end:");
        System.out.println(numbers);

        // Search
        if (numbers.contains(20)) {
            System.out.println("20 is found.");
        }

        // Get element
        System.out.println("Element at index 2: " + numbers.get(2));
        // Update
        numbers.set(2, 25);

        System.out.println("After updating:");
        System.out.println(numbers);

        // Remove
        numbers.remove(Integer.valueOf(25));

        System.out.println("After removing 25:");
        System.out.println(numbers);
        // Size
        System.out.println("Size: " + numbers.size());
    }
}
```

37. Expected Output

Original List:

[10, 20, 30]

After adding 5 at beginning:

[5, 10, 20, 30]

After adding 40 at end:

[5, 10, 20, 30, 40]

20 is found.

Element at index 2: 20

After updating:

[5, 10, 25, 30, 40]

After removing 25:

[5, 10, 30, 40]

Size: 4

38. Simple LinkedList Method Cheat Sheet

Method	Purpose	Example
add()	Add element	list.add(10);
addFirst()	Add at beginning	list.addFirst(10);
addLast()	Add at end	list.addLast(10);
add(index,value)	Insert at position	list.add(1,20);
get()	Get element	list.get(1);
set()	Update element	list.set(1,20);
remove(index)	Remove by index	list.remove(1);
remove(value)	Remove by value	list.remove("Ana");
removeFirst()	Remove first	list.removeFirst();
removeLast()	Remove last	list.removeLast();
contains()	Search	list.contains(20);
indexOf()	Find position	list.indexOf(20);
size()	Number of elements	list.size();
isEmpty()	Check if empty	list.isEmpty();
clear()	Remove everything	list.clear();

39. Operation Summary

The students can remember Linked List operations using this simple pattern:

CREATE

ADD

DISPLAY

ACCESS

UPDATE

SEARCH

DELETE

For Java:

Create:

LinkedList<Integer> list = new LinkedList<>();

Add:

list.add(10);

Display:

System.out.println(list);

Access:

list.get(0);

Update:

list.set(0, 20);

Search:

list.contains(20);

Delete:

list.remove(Integer.valueOf(20));